Lesson: Loops

CSE 4107 : Structured Programming I

Shahriar Ivan

Iteration Statements

- C's iteration statements are used to set up loops.
- A ***loop*** is a statement whose job is to repeatedly execute some other statement (the ***loop body***)
- In C, every loop has a ***controlling expression***
- Each time the loop body is executed (an ***iteration*** of the loop), the controlling expression is evaluated.
  - If the expression is true (has a value that's not zero) the loop continues to execute

Iteration Statements

- C provides ***three*** iteration statements:
  - The `while` statement is used for loops whose controlling expression is tested *before* the loop body is executed
  - The `do` statement is used if the expression is tested *after* the loop body is executed
- The `for` statement is convenient for loops that increment or decrement a counting variable

The while Statement

- Using a while statement is the easiest way to set up a loop.
- The while statement has the form

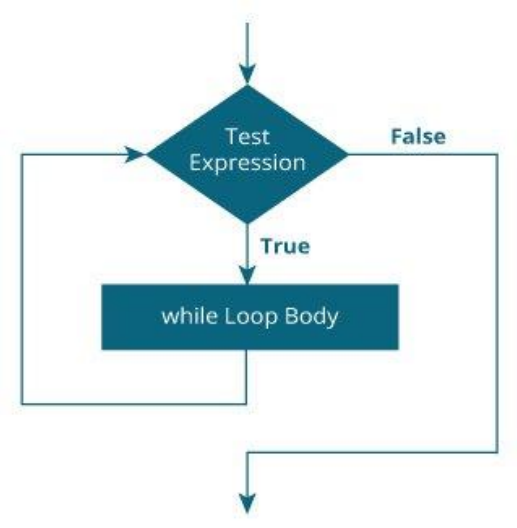
  while (*expression*) *statement*

- Example of a while statement:

```
while (i < n)    /* controlling expression */
    i = i * 2;       /* loop body */
```

- When a while statement is executed, the controlling expression is evaluated first

The while Statement



The while Statement

- If its value is nonzero (true), the loop body is executed and the expression is tested again
- The process continues until the controlling expression eventually has the value zero
- A while statement that computes the smallest power of 2 that is greater than or equal to a number *n*:

```
i = 1;
while (i < n)
        i = i * 2;
```

The while Statement

- A trace of the loop when n has the value 10:

i = 1;	i is now 1.
Is i < n?	Yes; continue.
i = i * 2;	i is now 2.
Is i < n?	Yes; continue.
i = i * 2;	i is now 4.
Is i < n?	Yes; continue.
i = i * 2;	i is now 8.
Is i < n?	Yes; continue.
i = i * 2;	i is now 16.
Is i < n?	No; exit from loop

The while Statement

- If multiple statements are needed, use braces to create a single compound statement:

```
while (i > 0) {
    printf("T minus %d and counting\n", i);
    i--;
}
```

- Some programmers always use braces, even when they're not strictly necessary:

```
while (i < n) {
    i = i * 2;
}
```

The while Statement

- Observations about the while statement:
  - The controlling expression is *false* when a while loop terminates. Thus, when a loop controlled by i > 0 terminates, i must be less than or equal to 0
  - The body of a while loop may not be executed at all, because the controlling expression is tested *before* the body is executed
  - A while statement can often be written in a variety of ways. A more concise version of the countdown loop:

```
while (i > 0)
  printf("T minus %d and counting\n", i--);
```

The while Statement

Infinite Loop

- A while statement won't terminate if the controlling expression always has a nonzero value
- C programmers sometimes deliberately create an ***infinite loop*** by using a nonzero constant as the controlling expression:

```
while (1) ...
```

- A while statement of this form will execute forever unless its body contains a statement that transfers control out of the loop (break, goto, return) or calls a function that causes the program to terminate

Programming Task

Printing a Table of Squares

- The program uses a while statement to print a table of squares.
- The user specifies the number of entries in the table:
- This program prints a table of squares.

```
Enter number of entries in table: 5
        1       1
        2       4
        3       9
        4       16
        5       25
```

Programming Task

Summing a Series of Numbers

- The program sums a series of integers entered by the user:

  This program sums a series of integers.

  Enter integers (0 to terminate): 8 23 71 5 0

  The sum is: 107

- The program will need a loop that uses scanf to read a number and then adds the number to a running total

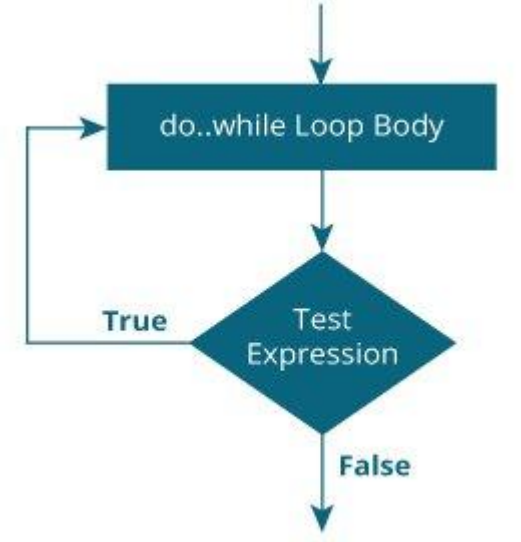
The do Statement

- General form of the do statement:

  do *statement* while (*expression*) ;

- When a do statement is executed, the loop body is executed first, then the controlling expression is evaluated
- If the value of the expression is nonzero, the loop body is executed again and then the expression is evaluated once more

The do Statement



The do Statement

- The countdown example rewritten as a do statement:

```
i = 10;
do {
        printf("T minus %d and counting\n", i);
        --i;
} while (i > 0);
```

- The do statement is often indistinguishable from the while statement
- The only difference is that the body of a do statement is always executed at least once

Programming Task

Calculating the Number of Digits in an Integer

- The program calculates the number of digits in an integer entered by the user:

  Enter a nonnegative integer: 60

  The number has 2 digit(s)

- The program will divide the user's input by `10` repeatedly until it becomes 0; the number of divisions performed is the number of digits
- Writing this loop as a do statement is better than using a while statement, because every integer—even 0—has at least one digit

The for Statement

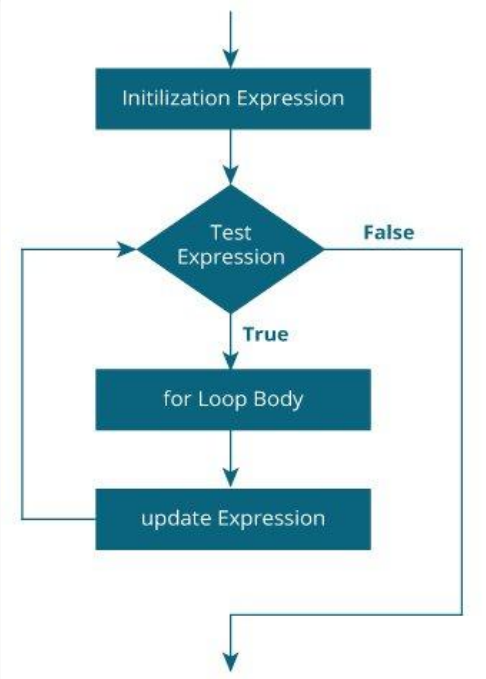
- The for statement is ideal for loops that have a “counting” variable, but it’s versatile enough to be used for other kinds of loops as well.
- General form of the for statement:

  for (*expr1* ; *expr2* ; *expr3*) *statement*

  - *expr1*, *expr2*, and *expr3* are expressions
- Example:

```
for (i = 10; i > 0; i--)
        printf("T minus %d and counting\n", i);
```

The for Statement



The for Statement

- Except in a few rare cases, a for loop can always be replaced by an equivalent while loop:

```
expr1;
while ( expr2 ) {
    statement
    expr3;
}
```

```
i = 10;
while (i > 0) {
    printf("T minus %d and counting\n", i);
    i--;
}
```

- *expr1* is an initialization step that's performed only once, before the loop begins to execute

The for Statement

- If i-- is replaced by --i, it does not have any change in i value as they are processed as statements

```
for (i = 10; i > 0; --i)
        printf("T minus %d and counting\n", i);
```

- The equivalent while loop shows that the change has no effect :

```
i = 10;
while (i > 0) {
        printf("T minus %d and counting\n", i);
        --i;
}
```

The for Statement

Statement Idioms

- The for statement is usually the best choice for loops that “count up” (increment a variable) or “count down” (decrement a variable)
- A for statement that counts up or down a total of n times will usually have one of the following forms:

Counting up from* 0 *to* n−1*:	`for (i = 0; i < n; i++) ...`
Counting up from* 1 *to* n*:	`for (i = 1; i <= n; i++) ...`
Counting down from* n−1 *to* 0*:	`for (i = n - 1; i >= 0; i--) ...`
Counting down from* n *to* 1*:	`for (i = n; i > 0; i--) ...`

The for Statement

Statement Idioms

- Common for statement errors:
  - Using < instead of > (or vice versa) in the controlling expression. “Counting up” loops should use the < or <= operator. “Counting down” loops should use > or >=
  - Using == in the controlling expression instead of <, <=, >, or >=
  - “Off-by-one” errors such as writing the controlling expression as i <= n instead of i < n

The for Statement

Omitting Expressions in a for Statement

- Any or All of the expressions that control a for statement to be omitted
- If the *first* expression is omitted, no initialization is performed before the loop is executed:

```
i = 10;
for (; i > 0; --i)
        printf("T minus %d and counting\n", i);
```

The for Statement

Omitting Expressions in a for Statement

- If the *third* expression is omitted, the loop body is responsible for ensuring that the value of the *second* expression eventually becomes false:

```
for (i = 10; i > 0;)
        printf("T minus %d and counting\n", i--);
```

- If *first* and *third* expressions are omitted, then it's the same as a while loop:

```
for (; i > 0;)
        printf("T minus %d and counting\n", i--);
```

is the same as,

```
while (i > 0)
        printf("T minus %d and counting\n", i--);
```

The for Statement

Omitting Expressions in a for Statement

- If the *second* expression is missing, it defaults to a true value, so the for statement doesn't terminate (unless stopped in some other fashion)
- For example, some programmers use the following for statement to establish an infinite loop:

```
for ( ; ; ) ...
```

The for Statement

for statements in C99

- In C99, the first expression in a for statement can be replaced by a declaration
- This feature allows the programmer to declare a variable for use by the loop:

```
for (int i = 0; i < n; i++)

...
```

- The variable *i* need not have been declared prior to this statement

The for Statement

for statements in C99

- A variable declared by a for statement can't be accessed outside the body of the loop (we say that it's not ***visible*** outside the loop):

```
for (int i = 0; i < n; i++) {
        ...
        printf("%d", i);        /* legal; i is visible inside loop */
        ...
}
printf("%d", i);                /*** WRONG ***/
```

The for Statement

for statements in C99

- If the program needs to access the variable after loop termination, it's necessary to use the older form of the for statement
- A for statement may declare more than one variable, provided that all variables have the same type:

```
for (int i = 0, j = 0; i < n; i++)
  ...
```

The for Statement

The Comma Operator

- More than one initialization expression or several increments
- Used in *first* and *third* expression
- General form

```
expr1, expr2
```

- Two step evaluation:
  - Evaluate `expr1` -> Discard value
  - Evaluate `expr2` -> Value of the entire expression
- `expr1` must have side effect -> Serves no purpose otherwise

The for Statement

The Comma Operator

- The comma operator is *left* associative, so the compiler interprets

  `i = 1, j = 2, k = i + j`

  as

  `((i = 1), (j = 2)), (k = (i + j))`

- Since the *left* operand in a comma expression is evaluated before the right operand, the assignments `i = 1, j = 2,` and `k = i + j` will be performed from left to right

Programming Task

Printing a Table of Squares (Revisited)

- Use `for` statement instead of while -> `square2.c`
- Expressions can be unrelated -> `square3.c`

Exiting from a Loop

The break Statement

- The normal exit point for a loop is at the beginning (as in a `while` or `for` statement) or at the end (the `do` statement).
- Using the `break` statement, it's possible to write a loop with an exit point in the middle or a loop with more than one exit point

Exiting from a Loop

The break Statement

- The `break` statement can transfer control out of a `switch` statement, but it can also be used to jump out of a `while`, `do`, or `for` loop.
- A loop that checks whether a number `n` is prime can use a `break` statement to terminate the loop as soon as a divisor is found:

```
for (d = 2; d < n; d++)
    if (n % d == 0)
        break;
```

Exiting from a Loop

The break Statement

- After the loop has terminated, an if statement can be use to determine whether termination was premature (hence n isn't prime) or normal (n is prime):

```
if (d < n)
        printf("%d is divisible by %d\n", n, d);
 else
        printf("%d is prime\n", n);
```

Exiting from a Loop

The break Statement

- We use the break statement in loops where the exit point is in the middle of the body
- Loops that read user input, terminating when a particular value is entered, fall into this category:

```
for ( ;  ; ) {
        printf("Enter a number (enter 0 to stop): ");
        scanf("%d", &n);
        if (n == 0)
            break;
        printf("%d cubed is %d\n", n, n * n * n);
 }
```

Exiting from a Loop

The break Statement

- When the statements are nested, the break statement can escape only one level of nesting
- Example:

```
while (...) {
        switch (...) {
                ...
            break;
                ...
        }
}
```

- break transfers control out of the switch statement, but not out of the while loop

Exiting from a Loop

The continue Statement

- The continue statement is similar to break:
  - break transfers control just past the end of a loop (control goes outside)
  - continue transfers control to a point just before the end of the loop body (control stays inside the loop)
- There's another difference between break and continue: break can be used in switch statements and loops (while, do, and for), whereas continue is limited to loops

Exiting from a Loop

The continue Statement

- A loop that uses the continue statement,

```
int n = 0, sum = 0, i;
while (n < 10) {
        scanf("%d", &i);
        if (i == 0)
                continue;
        sum += i;
        n++;
        /* continue jumps to here */
}
```

- Same loop, but without continue,

```
int n = 0, sum = 0, i;
while (n < 10) {
        scanf("%d", &i);
        if (i != 0) {
                sum += i;
                n++;
        }
}
```

Programming Task

Balancing a Checkbook

- Many simple interactive programs present the user with a list of commands to choose from
- Once a command is entered, the program performs the desired action, then prompts the user for another command
- This process continues until the user selects an “***exit***” or “***quit***” command

Programming Task (contd.)

Balancing a Checkbook

- The program, which maintains a checkbook balance, uses a loop of this type
- The user is allowed to clear the ***account balance***, ***credit money*** to the account, ***debit money*** from the account, ***display the current balance***, and ***exit*** the program

Programming Task (contd.)

Balancing a Checkbook (Sample Run)

```
*** ACME checkbook-balancing program ***
Commands: 0=clear, 1=credit, 2=debit, 3=balance, 4=exit

Enter command: 1
Enter amount of credit: 1042.56
Enter command: 2
Enter amount of debit: 133.79
Enter command: 1
Enter amount of credit: 1754.32
Enter command: 2
Enter amount of debit: 1400
Enter command: 2
Enter amount of debit: 68
Enter command: 3
Current balance: $1195.09
Enter command: 4
```

The Null Statement

- A statement can be ***null***—devoid of symbols except for the semicolon at the end.
- The following line contains three statements:

```
i = 0; ; j = 1;
```

- The null statement is primarily good for one thing: writing loops whose bodies are empty

The Null Statement

- Consider the following prime-finding loop:

```
for (d = 2; d < n; d++)
    if (n % d == 0)
        break;
```

- If the n % d == 0 condition is moved into the loop's controlling expression, the body of the loop becomes empty:

```
for (d = 2; d < n && n % d != 0; d++)
    /* empty loop body */ ;
```

- To avoid confusion, C programmers customarily put the null statement on a line by itself

The Null Statement

- Accidentally putting a semicolon after the parentheses in an if, while, or for statement creates a null statement.
- Example 1:

```
i = 10;
while (i > 0);{    /*** WRONG ***/
        printf("T minus %d and counting\n", i);
        --i;
}
```

- The extra semicolon creates an infinite loop

The Null Statement

- Example 2:

```
i = 11;
 while (--i > 0);              /*** WRONG ***/
       printf("T minus %d and counting\n", i);
```

- The loop body is executed only once; the message printed is:

```
T minus 0 and counting
```